Reflection Questions:

1- Which optimization produced the greatest improvement

The Bounded Thread Pool (ThreadPoolExecutor):
Switching from unmanaged "thread-per-client" model to a bounded thread pool produced the greatest improvement, it stablilized the system eliminated memory thread-exhaustion crashes.

2- Why is scalability important

It ensures that as the number of concurrent users grows, the system maintains predictable latency and throughput without choking or crashing.

3- What reliabiloty issues did you discover

1- Server-side race condition where a client could receive another client's Join/leave broadcast before its own AUTH-OK causing authentication to fail under concurrent load

2- Client-side race where auto-reconnect logic could try to update a GUI window that had already been destroyed during a server shutdown causing a crash

4- which optimization was the most difficult Diagnosing the authentication race condition. It didn't appear in normal single-client testing It only surfaced under concurrent load and required tracing to exact order of Socket registration versus message sending across multiple thread to find the root cause.

5. What additional improvements are required to support 100 users ?

A larger/ dynamic thread pool or a switch to Asynchronous I/O architecture . Instead of one thread per client, connection pooling or a proper message queue, moving from CSV / flat-file storage to real database for users and logs and horizontal scaling across multiple server instances behind a load balancer